Where to Stop:
Tile-Adaptive Early Exit in a Learned Image Decoder

Anonymous CVPR submission · Paper ID ****

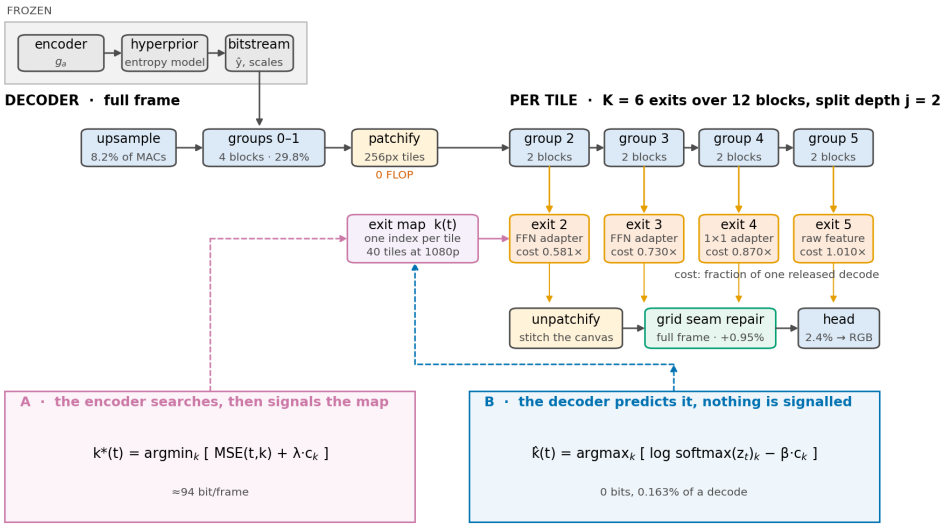

Figure 1. FLEX-UF end to end. Grey is frozen and never touched; blue is inherited from DCVC-UF and fine-tuned; orange and green are new. The first j block groups run over the whole frame, the rest run per tile on a shrinking active set, and the exit map that drives the shrinkage comes either from an encoder-side search (A) or from a decoder-side predictor (B).

Abstract

A learned image decoder spends the same computation on every frame regardless of what the frame contains. We add an early-exit ladder to the intra decoder of DCVC-UF, cut each frame into tiles, and let every tile leave the trunk at whichever of K exits is deep enough for that tile. The encoder is never touched and the coded payload is unchanged, so the method deploys against existing bitstreams. On the full 53-sequence HEVC/UVG/MCL-JCV common test set a 0.1 dB quality budget buys 32.3% of the decoder's multiply-accumulates at the lowest rate and 16.8% at the highest, at a BD-Rate cost of 0.88%. Two findings shape the design. First, tiling is not free: a 3×3 convolution at a tile border meets padding instead of a neighbour, and on this decoder that costs 0.677 dB at q63 under the stock rule — several times the entire budget, spent before a single tile has exited early. The two remedies that work cost nothing, a higher-order border estimator is *worse* than a zeroth-order one, and a learned repair module cannot pay for itself. Second, a quality budget only buys compute inside a bounded window: below a *floor* set by tiling no allocation is feasible, and above a *saturation* point every tile already sits on the cheapest rung. We measure both ends at every rate. We also show that the obvious implementation of the ladder is *slower* than the dense decoder, and that a bit-identical reordering of the per-tile loop recovers 28.5% of wall-clock against a 35.3% arithmetic prediction.

1. Introduction

Learned codecs are compared on rate and distortion, with complexity reported as a single number: so many GMAC per frame, so many milliseconds. That number is a constant. A learned decoder runs the same graph on a page of text and on a cloudless sky, and the sky is not harder.

Classification networks abandoned this a decade ago. Early-exit architectures attach classifiers at intermediate depths and stop once the prediction is confident [3, 15, 20]. Super-resolution adopted the idea spatially: ClassSR [12] routes patches to networks of different capacity, APE [1] exits patches at different depths. Learned compression went another way — slimmable autoencoders [22, 23] give one model several complexity levels, but the level is chosen *per stream*, not per region, and switching it changes the bitstream.

We ask the spatial-adaptivity question inside a learned decoder under a constraint that makes the answer deployable: **the encoder is frozen and the coded payload is unchanged.** That rules out retraining the analysis transform, changing the entropy model or altering the latent, and leaves exactly one place to spend adaptivity — the synthesis transform.

FLEX-UF is an exit ladder over the twelve residual blocks of the DCVC-UF intra decoder. The first j blocks run over the whole frame; the rest run per tile on a shrinking set, and that shrinkage is the saving. Each exit reaches the shared head through a small pointwise adapter, zero-initialised so that at step zero the deepest exit reproduces the released decoder bit-exactly.

Making it work depended less on the ladder than on two things that have little to do with early exit as usually studied.

Tiling has a price, and it is large.

Spatial adaptivity needs tiles, and the moment a frame becomes tiles every 3×3 convolution at a border reads invented values. The error is 0.677 dB at high rate under the stock padding rule — several times the budget, paid before any tile has saved an operation. Section 4 treats padding as an *estimator* of the unseen neighbour and measures four; the ordering is not the obvious one.

A quality budget has a working range.

Because tiling costs something even when nothing exits early there is a *floor*: budgets below it admit no allocation. Because the ladder has a shallowest rung there is a *saturation* point above which more quality buys nothing. The working 0.1 dB budget uses 45% of that window at the lowest rate and 9% at the highest.

Contributions.

(i) A tile-adaptive early-exit ladder for a learned image decoder that leaves the encoder and coded payload untouched, saving 32.3% to 16.8% of decoder MACs for 0.1 dB on the full CTC set. (ii) The floor/saturation characterisation of a quality budget, with both ends measured, and the observation that the architectural ceiling is reachable at low rate. (iii) A quantitative account of the tiling penalty, including two negative results. (iv) Two allocation regimes — an encoder-side search signalling a ~79-bit map, and a decoder-side predictor signalling nothing — with bits and compute charged on both sides. (v) A wall-clock result: the obvious implementation is slower than the dense decoder, and a bit-identical reordering recovers 28.5%.

2. Related work

Early exit.

BranchyNet [20] and MSDNet [3] established intermediate classifiers with a confidence rule; SDN [15] framed it as mitigating overthinking. Joint training of all exits follows Scardapane et al. [18]; distillation between exits follows Phuong and Lampert [17], with the caveat from [19] that too large a student-teacher gap hurts the shallowest exits — which is why ours is between *adjacent* exits. All of this exits on a confidence signal computed from the network's own output. A decoder has no such signal: the quantity that would decide is the error against a source it cannot see.

Spatially adaptive inference.

ClassSR [12] sorts patches into easy/medium/hard; APE [1] exits patches at different depths; Glance-and-Focus [8] spends resolution adaptively. These share our per-region premise and share the tiling problem, though the cost of tile borders is rarely quantified. The closest prior work on the border itself is the per-channel AR(1) padding of Kaseva et al. [11], which we implement and evaluate.

Complexity control in learned compression.

SlimCAE [22] and slimmable video codecs [23] expose several widths of one model; the choice is per stream and changes the encoder, so the bitstream is not interchangeable. DCVC-FM [13] and DCVC-UF [14] reduce cost architecturally, for every frame equally. Our axis is orthogonal: model fixed, bitstream fixed, only *how much of the decoder runs where* varies.

Signalling versus prediction.

That a decoder is *told* a mode decision rather than inferring it is the norm in standardised video coding: HEVC [9] and VVC [21] transmit partitioning, prediction mode and transform tree. We evaluate both and treat the signalled variant as the conventional design. The nearest neighbour in compression is spatial competition [27], which selects among several codecs per region and signals a mode map: the structure of the side information is ours, the axis is not — they select which network at fixed complexity for a rate gain, we select how much of one network at fixed rate for compute.

Allocating a budget over units.

The construction we use is not new and should not be presented as such. Shoham and Gersho [28] showed that for a finite set of per-unit operating points a Lagrangian sweep decouples the allocation across units and traces exactly the lower convex hull of the achievable set; Ortega and Ramchandran [29] made it standard practice in image and video coding. What we add is the structure this particular operating set has — a floor below which no allocation is feasible, a saturation point above which none improves, and a measurement of what the convex-hull restriction costs. The same relaxation has resurfaced for test-time compute in language models [30], with per-instance decoupling and a binary search on the multiplier: the identical structure in a domain with no rate axis.

Tile boundaries.

Every method that processes an image in independently-computed tiles meets the same artefact, and the remedies in the literature are overlap and averaging, local padding from neighbouring patches [31], training with overlaps [32], and fitted extrapolation [11]. Local padding is closest to the exact remedy of Section 4.4, and the difference is accounting: it pads every convolutional layer and does not report the cost, while we pad only the 0.29% of each block that has any spatial extent — which is what makes the exact fix cost 0.032% of the decode rather than a multiplier on all of it.

Where the time goes.

DCVC-RT [33] argues that operational rather than computational complexity is the speed bottleneck for neural codecs, evidenced by channel reductions that yield linear rather than quadratic speedups. Section 5.6 is a sharp instance of that claim inside one loop: a 20% reduction in operations produced a 10.9% *slow-down* until the loop was reordered, with the arithmetic untouched.

3. Method

3.1 Where the computation is

The DCVC-UF intra decoder is one upsampling block, twelve DepthConvBlocks and a head, costing 453.5 GMAC per 1080p frame. The twelve blocks are 89.4% of it, which is why the ladder is built across them. Inside one block at C=384, the only operator with any spatial extent is a 3×3 depthwise costing 9C against the block's $8C^2+9C$ — **0.29%.** That number recurs throughout: it is why tile borders damage the picture, why our adapters are pointwise on purpose, and why the exact remedy of Section 4.4 is affordable.

3.2 The exit ladder

With K exits over the twelve blocks and split depth j, groups 0..j−1 run full frame for every tile and groups j..K−1 run per tile. A tile assigned exit k runs groups j..k and leaves. Writing c_k for the cost of exit k in units of one released decode, a frame with exit map k costs the mean of c over its tiles.

Two consequences are easy to state wrongly. Exits shallower than j are indistinguishable, so the usable ladder has K−j distinct costs, and the *architectural ceiling* is S_max = 100(1−c_j)%, attained when every tile takes exit j. For the shipped setting (K=6, j=2, 256 px tiles) S_max = 41.9%. Section 5.4 shows this bound is *reached* at low rate, which makes it an operating point rather than an asymptote.

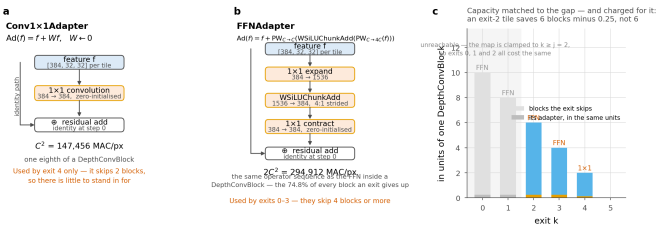

Figure 2. Inside an exit adapter. Both kinds are entirely pointwise, so no adapter adds receptive field and none contributes any tile-boundary penalty. Capacity is matched to the number of blocks the exit skips, and charged for: an exit-2 tile saves six blocks minus 0.25, not six.

3.3 Exit adapters

An early exit hands the shared head a feature the head was not fitted to; the adapter is the correction. We use a residual 1×1, Ad(f) = f + Wf with W zero-initialised, for exits that skip little, and the pointwise expand/activate/contract pair of the block's own FFN for exits that skip four blocks or more. Both are *pointwise by design*: a 3×3 inside an adapter would add seam damage precisely at the tiles that took an early exit, the ones least able to afford it. Zero-initialising the last layer makes every adapter the identity at step zero, so the deepest exit is bit-exactly the released decoder before training begins.

3.4 Allocation

Given per-tile distortions D(t,k) and costs c_k, the allocation minimising distortion at a compute budget is the Lagrangian **k*(t) = argmin_k [D(t,k) + λ·c_k]**, with λ bisected so the frame lands on the budget. Both quantities are available *to the encoder*, which holds the source. This is configuration **A**: the encoder searches and transmits the map, which entropy-codes to ~79 bits per 1080p frame, 0.021% of a typical bitrate. It costs the encoder about one extra decode and the decoder nothing.

The decoder cannot evaluate it: D(t,k) is the error against a source it never sees. This is not a hard estimation problem, it is a missing variable, and no architecture removes it. Configuration **B** therefore predicts: a 144K-parameter head reads the stem feature, the decoded latent, the entropy model's scales and the quality index, and decides **k(t) = argmax_k [log softmax(z_t)_k − β·c_k]**, with β bisected as λ is. Nothing is added to the file. The head is trained against a *frozen* decoder with a cost-sensitive cross-entropy to the oracle's choice, each tile weighted by the regret of choosing wrongly, plus loss-free load balancing [16] biased toward the oracle's own exit distribution rather than toward uniform.

3.5 Training

All exits are decoded every step and the objective is L = L_RD + w_a·L_anchor + w_d·L_distill. L_RD is the released rate-distortion loss with MSE averaged over exits. L_anchor pins the deepest exit to the released decoder, so the reference every saving is quoted against cannot drift underneath the measurement. L_distill supervises adapters in *feature* space, exit k imitating exit k+1 — feature space because the head is a fixed map from feature to RGB and matching the deeper feature is the stronger constraint, 384 dense channels of target instead of 3; adjacent rather than deepest for the reason in [19].

Critically, the adapters are trained *through the tiled decode path they are deployed in*. Training them full-frame and tiling only at inference loses 0.14–0.24 dB; training through the deployed path gains 0.51–0.90 dB. The sign of the effect flips.

4. The price of tiles

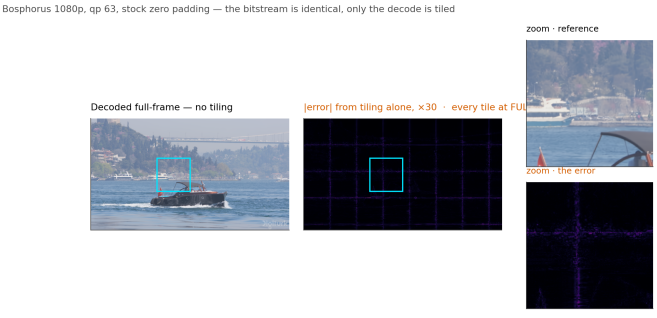

Figure 3. The artefact, before anything is done about it. One frame decoded twice from the *same* bitstream with the same weights, every tile at full depth — no early exit anywhere. The only difference is that the right-hand decode was tiled. The error map is the tile lattice and nothing else.

A 3×3 depthwise computes a weighted sum over its neighbours. Decoded full frame those neighbours exist; decoded per tile they do not, and the kernel is handed whatever the padding rule invents. Each convolution extends the contaminated region by one ring, so with b per-tile blocks on a tile of side F the fraction of the tile within reach of an invented value is 1 − ((F−2b)/F)², which is 0.750 at the shipped F=32, b=8: not a thin border but a structured error across most of the tile.

That fraction says which pixels are affected, not how badly, and it is not what governs the penalty. Sweeping the split depth sweeps b from 12 to 0, with b=0 as an exact zero-seam control, and the measured seam follows a power law: **seam** ∝ $\mathbf{b}^{\alpha}$ with α = 2.38, 2.22 and 1.93 at q0, q32 and q63, r = 0.98–0.995 in log-log. Fitted with one free scale the area fraction is wrong by 160–264% where the power law is wrong by 12–22%. The exponent near two has a reading: the count of contaminated pixels grows like perimeter times depth, and the error accumulated in each grows with how many convolutions reached it. The area law saturates once every pixel is touched; the penalty does not.

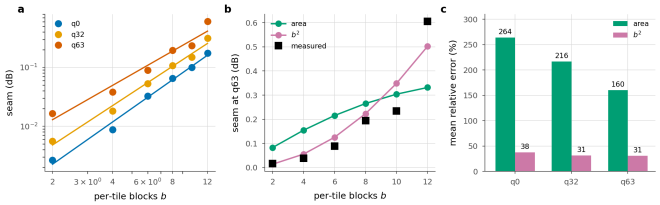

Figure 4. The seam against per-tile depth. Sweeping the split depth sweeps b; b=0 is an exact control and measures 0.0000 dB at all three rates. **a**, the measurement with the fitted power law. **b**, both models against q63 with one free scale each. **c**, mean relative error. The area fraction saturates once every pixel is contaminated; the penalty does not.

4.1 Padding is an estimator

The useful way to see border padding is as an *estimator* of the unseen neighbour, whose error is the seam. Table 1 measures four, with early exit switched off so tiling is the only difference from a full-frame decode.

Border estimator	q0	q32	q63
Zeros (stock)	0.126	0.287	0.677
Replicate	**0.071**	**0.123**	**0.218**
Linear extrap.	0.198	0.286	0.384
AR(1) per channel~citearls	0.061	0.107	0.197

Table 1. Border estimators, dB below the released decoder on the same latent, 256 px tiles, full CTC. Lower is better.

Two results deserve emphasis. **A higher-order estimator is worse.** Extrapolating the local gradient past a boundary amplifies whatever noise sits on it; assuming local constancy does not. Guessing harder is not guessing better, and the gap is large: 0.384 dB against 0.218 dB at q63. **The best estimator loses on cost.** The per-channel AR(1) fit of [11] reaches 0.197 dB, about 0.02 dB better than replication, for 10.7% of decode wall-clock. Against a 0.1 dB budget and a ~24% saving that trade does not close, and we drop it.

4.2 What actually removes the seam

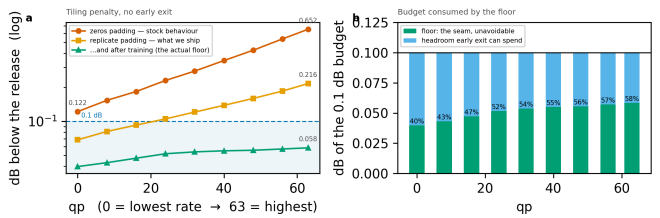

Figure 5. The tiling penalty across the whole rate range, every tile at full depth. The two steps that matter cost nothing, and the largest single factor is training the ladder with the seam present. Right: the floor is charged *inside* the quality budget and consumes a growing share of it.

Tile size is free — a tiled decode's MAC count does not depend on it at all — and the damage scales with the tile perimeter (Table 2). What it costs instead is routing granularity: 40 tiles per 1080p frame at 256 px against 160 at 128 px.

Estimator, q63	128 px	256 px	ratio
zeros	1.249	0.677	×0.54
replicate	0.372	0.218	×0.58
arls	0.332	0.197	×0.59

Table 2. Tile size, dB at q63. Doubling the side roughly halves the penalty, as the contamination law predicts, and costs no computation.

The largest factor of all is easiest to overlook. Replicate padding leaves 0.218 dB at q63 on the untrained ladder; after training the same configuration leaves 0.072 dB. More than two thirds of what the estimator could not fix is absorbed by weights learning to live with it. No module in this paper removes as much.

4.3 A learned repair module, and why we reject it

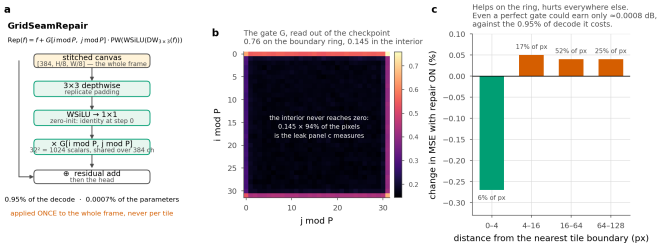

Figure 6. Grid seam repair. A correction gated by position within a tile, applied once to the stitched frame. The trained gate learns the boundary ring (0.76) but never switches off inside (0.145 over 94% of pixels), and the measured effect follows: it helps on the ring and hurts everywhere else.

Since the tile lattice is known exactly at training and inference, a repair can be *told* where to look rather than having to infer it: Rep(f) = f + G[i mod P, j mod P]·PW(WSiLU(DW3×3(f))), with G a P×P gate shared over channels, initialised at exp(−d/τ). It costs 0.95% of the decode.

It does not earn that. Splitting per-pixel error by distance from the nearest tile boundary, the module gains 0.27% in the 0–4 px band (6.2% of pixels) and loses 0.04–0.05% everywhere else. Even with a *perfect* gate — zero correction in the interior, the boundary gain unchanged — the ceiling on what it could earn is ≈0.0008 dB for 0.95% of the decode. We rejected AR(1) padding at 0.0019 dB per point of decode; this is worse by an order of magnitude. Tightening the gate cannot rescue it, because there is almost nothing left to win.

4.4 Removing the cause, and why it does not help

Because only 0.29% of each block has spatial extent, the exact fix is affordable: give the 3×3 its real neighbour from a shared canvas, for +0.032% of the decode. And it is exact — at uniform depth a coupled tiled decode is bit-identical to a full-frame one, once the comparison is not confounded by the repair module, which runs on the stitched canvas and has no full-frame counterpart. Measured: 1.13e-2 with the repair on, **exactly 0** with it off, against 6.06e-2 for replicate padding.

It also removes most of the floor. Switched on at inference the floor falls from 0.036 to 0.003 dB at q0 and from 0.056 to 0.030 at q63, i.e. 91% and 47% of the tiling penalty.

And it destroys the allocation. At the same 0.1 dB budget the saving falls from 25.9% to 4.2% at q32 and from 19.3% to 0.4% at q63.

The reason is the condition in the exactness statement. Coupling is exact when every tile is at the *same* depth, and routing is the deliberate violation of that condition. With replicate padding a tile is entirely independent of its neighbours, so the allocation may give adjacent tiles any depths it likes; coupling makes a tile depend on its neighbours, and under routing those neighbours ran a different number of blocks. A shallow tile reading a deep neighbour's activation is a configuration the weights have never seen, and that mismatch costs far more than the seam it removed. We report it because the natural reading of an exactness result — adopt the exact fix — is the wrong one here, and the tension is a property of the combination that any spatially adaptive decoder inherits.

5. Experiments

Setup.

The decoder is fine-tuned on OpenImages 512×512 crops with the encoder frozen (max|Δ| = 0 asserted every run), one λ drawn per sample from a log-spaced range covering all 64 quality indices, so a single set of weights serves the whole rate range. Evaluation is on the 53 sequences of the common test set — UVG, MCL-JCV and HEVC classes B, C, D and E — one intra frame each.

Measurement protocol.

Two details change the numbers enough to state. First, the per-tile distortion table must be built on the *deployed* decode path — one tiled decode per exit — and not by tapping the exits of a single full-frame forward pass. The latter is the natural implementation and correct for training, but with a full-frame reference on the other side of the ratio the tiling penalty cancels and the reported quality belongs to a decoder nobody ships. On our model the difference is +0.035 dB at the deepest exit and +0.008 dB at the shallowest; it grows with how many blocks ran per tile, so it cannot be corrected after the fact. Second, after the allocation is chosen we decode that *mixed* map once and report the distortion it actually produces.

Reporting conventions.

Every dB is measured against the *released* decoder's full-frame decode of the *same* latent, and our side is the deployed tiled decode, so the tiling penalty is inside every number. Savings are fractions of the released decoder's cost; our own deepest exit costs 1.0095 of it, and using that as the denominator would flatter every result by 0.6–0.8 points.

5.1 Main result

Figure 7. What the saving looks like. The same bitstream decoded by the released decoder and by ours at the 0.1 dB operating point, 30.5% fewer multiply-accumulates. The crop is the tile that gave up the most quality, chosen automatically, so this is the method's worst case on this frame rather than a flattering one.

Budget	q0	q16	q32	q48	q63	mean
0.10 dB	32.3	27.6	22.5	19.8	16.8	**23.8**
0.30 dB	41.9	41.9	41.9	40.6	38.4	**41.0**
0.50 dB	41.9	41.9	41.9	41.9	41.9	**41.9**

Table 3. Decoder MACs saved (%) against the released decoder, per quality index and budget, configuration A. The 0.3 and 0.5 dB rows sit on the architectural ceiling almost everywhere: past that point a looser budget buys nothing.

At 0.1 dB the method saves 32.3% at the lowest rate and 16.8% at the highest, averaging 23.8%, at a BD-Rate cost of 0.88% — that is, the compute is worth about the same as a 0.88% increase in bitrate. The fall

with rate is not an artefact of the ladder: high-rate reconstructions carry detail the shallow exits cannot reproduce, and the floor rises with rate.

Decoder	GMAC	% of release	ms
Released DCVC-UF	454	100.0	274
FLEX-UF, all tiles deepest	458	101.0	297
FLEX-UF, 0.1 dB budget	346	76.2	196
FLEX-UF, architectural ceiling	263	58.1	–

Table 4. Decoder complexity at 1080p. Our deepest exit costs slightly more than the release because it still pays the seam-repair module; that 1.0095, not 1.0, is what every saving here is *not* divided by.

5.2 Is per-tile adaptivity necessary?

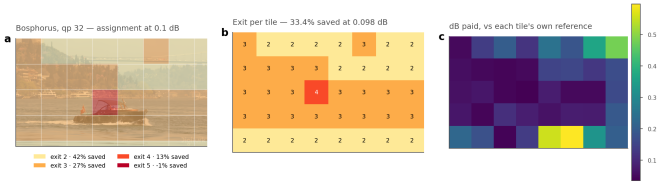

Figure 8. Where the decoder spends. (a) the assignment overlaid on the frame, (b) the exit index per tile, (c) the quality each tile gives up against its *own* full-depth reference. Water and sky leave at the shallowest rung; the boat and shoreline run deep.

q	Allocation	DeltaPSNR (dB)	MACs saved (%)
0	uniform, exit 2†	0.179	41.9
	uniform, exit 3	0.093	27.0
	uniform, exit 4	0.059	13.0
	uniform, exit 5	0.036	-1.0
	random (matched mix)	0.125	32.4
	rate-ranked (free)	0.107	32.4
	oracle	**0.100**	**32.4**
16	uniform, exit 2†	0.219	41.9
	uniform, exit 3†	0.118	27.0
	uniform, exit 4	0.075	13.0
	uniform, exit 5	0.045	-1.0
	random (matched mix)	0.133	27.5
	rate-ranked (free)	0.106	27.5
	oracle	**0.100**	**27.5**
32	uniform, exit 2†	0.282	41.9
	uniform, exit 3†	0.147	27.0
	uniform, exit 4	0.090	13.0
	uniform, exit 5	0.054	-1.0
	random (matched mix)	0.141	22.5
	rate-ranked (free)	0.107	22.5
	oracle	**0.100**	**22.5**
48	uniform, exit 2†	0.333	41.9
	uniform, exit 3†	0.176	27.0
	uniform, exit 4†	0.103	13.0
	uniform, exit 5	0.062	-1.0
	random (matched mix)	0.145	19.7
	rate-ranked (free)	0.106	19.7
	oracle	**0.100**	**19.7**
63	uniform, exit 2†	0.396	41.9
	uniform, exit 3†	0.206	27.0
	uniform, exit 4†	0.116	13.0
	uniform, exit 5	0.072	-1.0
	random (matched mix)	0.141	16.7
	rate-ranked (free)	0.110	16.7
	oracle	**0.099**	**16.7**

Table 5. Adaptivity against three controls at 0.1 dB. † marks uniform depths exceeding the budget. Random and rate-ranked draw the oracle's own exit histogram — same average cost, same mix of depths — and differ only in which tile gets which depth.

The uniform rows answer the obvious alternative, and the answer sharpens with rate. At the lowest rate a static decoder reaches exit 3 within the budget and saves 27.0%, against the oracle's 32.3%. At q48 and q63 the only uniform depth that fits is the deepest, which *costs*

0.95% rather than saving anything — so there the choice is not between 17% and less, it is between 17% and nothing. Averaged over rates the best static allocation saves 10.2% where the adaptive one saves 23.8%.

The two shuffled rows separate effects that are easy to conflate. Given the oracle's histogram but assigned at random, quality collapses: what the allocation buys is not that some tiles run shallower, it is knowing *which* ones can. The rate-ranked row keeps that histogram and orders it by a signal the decoder already holds — the bits the entropy model spent on each tile. It is the cheapest conceivable router, with no parameters and no training, and it is the natural analogue of the confidence rules early-exit classifiers use.

It recovers most of the gap. At q63, random costs 0.141 dB and the oracle 0.100 dB at identical compute; rate-ranking costs 0.110 dB, i.e. 75% of the oracle's advantage over chance, at 0.68–0.79 agreement with the oracle's map. A learned router therefore has to beat a free baseline already three quarters of the way there — a comparison we would not have made without this control, and one we suggest any adaptive-inference paper should report.

5.3 Resolution, and the granularity of a tile

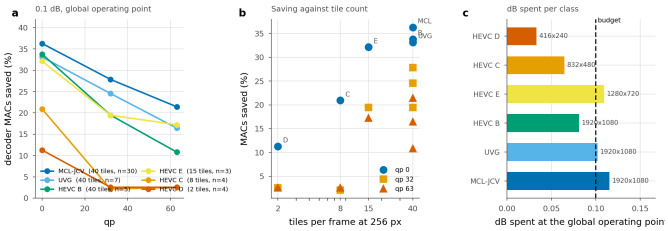

Figure 9. Saving by test class at one global operating point. λ is bisected once so the whole set lands on the budget, exactly as it would be in deployment; each class is then reported at that λ. The ordering follows tile count, not content.

Class	Resolution	Tiles	n	q0	q32	q63
MCL-JCV	1920x1080	40	30	36.3	27.9	21.4
UVG	1920x1080	40	7	33.1	24.5	16.4
HEVC_B	1920x1080	40	5	33.8	19.5	10.8
HEVC_E	1280x720	15	3	32.1	19.4	17.2
HEVC_C	832x480	8	4	20.9	2.1	2.5
HEVC_D	416x240	2	4	11.3	2.5	2.5

Table 6. Saving by class (%) at a 0.1 dB budget set globally over all 53 sequences. "Tiles" is how many 256 px tiles a frame of that resolution contains.

Completing the test set exposed a dependence the 1080p-only subset had hidden. At 1080p, where a frame is 40 tiles, the method saves 36.3% (MCL-JCV), 33.1% (UVG) and 33.8% (HEVC B) at the lowest rate — three classes of very different content within three points of each other. At 832×480 it is 8 tiles and 20.9%; at 416×240 it is 2 tiles and 11.3%. At q63 the two low-resolution classes fall to 2.5% against 21.4% for 1080p. The method is a function of tile count far more than of content.

The cause is granularity: with two tiles per frame there is almost no allocation to make and the Lagrangian degenerates toward a uniform choice. The fix is not more compute but a tile size chosen relative to the frame — the tiling penalty scales with the tile perimeter and the MAC count does not depend on tile size at all, so the trade is between seam and granularity and should be resolved per resolution. We report the single 256 px setting used throughout and note that a resolution-adaptive tile size is the obvious extension.

5.4 The working range of a quality budget

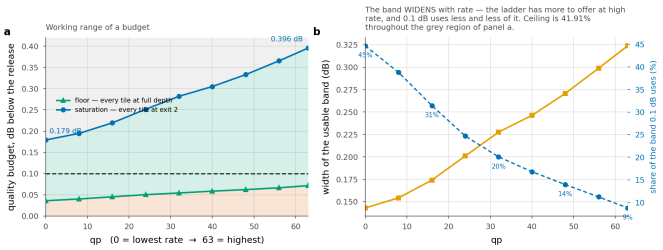

Figure 10. Three regions, and only the middle one is a design choice. Below the floor no allocation meets the budget; above saturation every tile is already on the cheapest rung. The working 0.1 dB budget uses 45% of the window at the lowest rate and 9% at the highest.

q	0	16	32	48	63
Floor D_min	0.036	0.045	0.054	0.062	0.072
Saturation D_mathrmsat	0.179	0.219	0.282	0.333	0.396
Usable band	0.143	0.174	0.228	0.271	0.324
0.1 dB uses	45%	31%	20%	14%	9%

Table 7. Floor and saturation, dB below the released decoder. The floor is what tiling costs with no early exit at all; saturation is where every tile reaches exit j and the ceiling is attained.

Two numbers bound what any budget can do. The **floor** is the distortion of a tiled decode with every tile at full depth — pure tiling penalty, 0.036 dB at q0 rising to 0.072 dB at q63. A budget below it admits no allocation. The **saturation** point is the distortion when every tile takes exit j; any budget at or above it attains the ceiling and a larger one attains nothing more.

A consequence worth stating plainly: at q0 the ceiling is reached at 0.179 dB. The architectural bound is not an asymptote, it is an operating point, and beyond it the limiting factor is the *ladder*, not the budget — an argument for a finer ladder rather than a looser budget. It also identifies a narrow regime: budgets in [0.179, 0.194) dB saturate the lowest rate and nothing else, a window 15 millibels wide.

5.5 Signalled versus predicted allocation

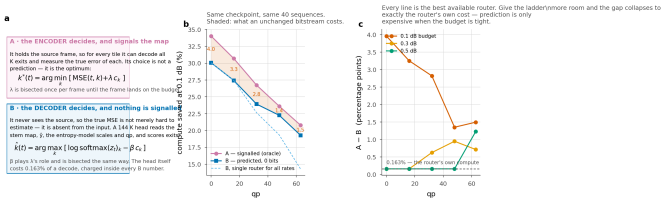

Figure 11. Who decides. A signals the oracle map at ~79 bits/frame; B predicts it from decoder-side data only and signals nothing. The gap collapses to the router's own compute once the budget is loose enough that both saturate.

Configuration A is exact by construction and costs bits; B is approximate and costs none. The comparison is not a wash in either direction: the prediction gap is real at a tight budget and vanishes at a loose one, because once the budget saturates both configurations send every tile to the same rung and the only remaining difference is the predictor's own 0.163% of decode.

5.6 Complexity and wall-clock

q	Released (ms)	Routed sorted (ms)	Saved (%) measured	MACs
0	274	196	**28.5**	35.3
32	401	314	**21.8**	28.0
63	512	438	**14.4**	20.1

Table 8. Wall-clock, 1080p, median of 40 interleaved iterations, at the 0.1 dB operating point. "MACs" is what the arithmetic predicts; "measured" is the sorted per-tile loop.

A saving in multiply-accumulates is not a saving in time, and here the gap was initially total. Tiling costs 8.5% before anything exits early, and the obvious implementation of a shrinking active set — a boolean mask, a gather of survivors and a scatter of the finished at every group boundary — forces a device-to-host synchronisation per boundary. Profiling attributes 32% of the per-tile loop to that bookkeeping. Measured end to end, the routed decoder was *slower* than the dense one at high rate despite doing 20% fewer operations.

Sorting the tiles once by exit depth removes it. Descending, "still active at group g" becomes a contiguous prefix: each group is a slice rather than a gather, the boundaries come from one cumulative count rather than a mask per group, and the finished tiles return through the inverse permutation in a single scatter. The arithmetic is unchanged and the output is bit-identical on GPU. The saving at q0 goes from 9.9% to 28.5% against a 35.3% arithmetic prediction.

We report this because the negative result is the more useful half: a paper reporting only MACs would have claimed a speedup that the same code, run as written, did not deliver.

5.7 The right ladder depends on the budget

Ladder	Config	Ceiling	0.1 dB	0.3 dB	0.5 dB
RECIPE512	K=6, j=2, 256 px	41.9	23.8	41.0	41.9
BEST	K=6, j=2, 256 px	41.9	**24.2**	38.5	41.3
BEST128	K=6, j=2, 128 px	41.9	19.3	36.6	40.6
FINE12	K=12, j=4, 128 px	50.3	19.0*	**42.0**	**48.6**

Table 9. Ladder configurations, mean saving (%) over the five rates, same test set and protocol. * one rate is infeasible at that budget — the ladder's floor exceeds it — so the mean is over the remaining four.

Section 5.4 argued that once a budget saturates a ladder the only way to spend more is a rung that does not exist. Table 9 measures it. A finer ladder (K=12, j=4) has a ceiling of 50.3% against 41.9%, and the orderings cross between 0.1 and 0.3 dB. At 0.1 dB the coarse ladder wins, and the fine one cannot even reach the budget at the highest rate: splitting later puts twice as many blocks in the per-tile section, which raises the floor. At 0.5 dB the fine ladder wins by 6.7 points, 48.6% against 41.9%, because the coarse one has been pinned at its ceiling since 0.3 dB.

So the ladder is not a hyperparameter to be tuned once. It is a function of the operating point, and the floor-saturation window is what tells you which side of the crossover you are on.

6. Limitations

The seam is reduced, and the exact remedy is not usable as it stands. Canvas coupling removes 47–91% of the floor and is bit-exact at uniform depth, yet collapses the routed saving because routing puts neighbouring tiles at different depths. Whether a decoder trained with coupling recovers both at once is open, and it is the experiment we would run next. Until it exists the floor is a cost this method pays.

Intra frames only. This is the image path of a video codec. Extending the ladder to inter frames raises a question this paper does not answer: an exit map propagates through the reference chain, so a shallow tile in one frame is a worse reference for the next.

Training is not converged. The measured saving is still increasing at every checkpoint measured more than once, so these numbers are a lower bound.

A single decoder. All results are on DCVC-UF's intra decoder. Nothing in the method is specific to it — the ladder needs only a residual trunk with a shared head — but that is an argument, not a measurement.

7. Conclusion

Learned decoders spend a constant amount of computation on a non-constant world. An early-exit ladder over the tiles of a frame recovers a substantial fraction of it — 32.3% to 16.8% of decoder MACs for a 0.1 dB budget — without touching the encoder or the coded payload.

The two lessons we would carry to any spatially adaptive decoder are the ones that were not about early exit. Tiling is expensive, its cost is dominated by a single 3×3 that is 0.29% of the arithmetic, and the remedies that work are geometric rather than learned. And a quality budget is only a control variable inside a measurable window: below the floor it admits nothing, above saturation it buys nothing, and reporting a saving without stating where in that window it sits leaves the most useful part of the result out.

References

[1] S. Wang et al. Adaptive patch exiting for scalable single image super-resolution. ECCV, 2022.

[2] J. Ballé et al. Variational image compression with a scale hyperprior. ICLR, 2018.

[3] G. Huang et al. Multi-scale dense networks for resource efficient image classification. ICLR, 2018.

[4] G. Bjøntegaard. Calculation of average PSNR differences between RD curves. VCEG-M33, 2001.

[5] B. Bross et al. Overview of the Versatile Video Coding standard. IEEE TCSVT, 2021.

[6] Z. Cheng et al. Learned image compression with discretized Gaussian mixture likelihoods. CVPR, 2020.

[7] W. Fedus et al. Switch Transformers. JMLR, 2022.

[8] G. Huang et al. Glance and Focus networks for dynamic visual recognition. IEEE TPAMI, 2023.

[9] G. J. Sullivan et al. Overview of the High Efficiency Video Coding standard. IEEE TCSVT, 2012.

[10] E. Jang et al. Categorical reparameterization with Gumbel-softmax. ICLR, 2017.

[11] T. Kaseva et al. Per-channel autoregressive linear prediction padding in tiled CNN processing of 2D spatial data. arXiv:2502.12300, 2025.

[12] X. Kong et al. ClassSR: a general framework to accelerate super-resolution networks by data characteristic. CVPR, 2021.

[13] J. Li, B. Li, Y. Lu. Neural video compression with feature modulation. CVPR, 2024.

[14] J. Li, B. Li, Y. Lu. Uniformly accelerated neural video codec with feature refinement. ACM MM, 2024.

[15] Y. Kaya et al. Shallow-deep networks: understanding and mitigating network overthinking. ICML, 2019.

[16] L. Wang et al. Auxiliary-loss-free load balancing strategy for mixture-of-experts. arXiv:2408.15664, 2024.

[17] M. Phuong, C. H. Lampert. Distillation-based training for multi-exit architectures. ICCV, 2019.

[18] S. Scardapane et al. Why should we add early exits to neural networks? Cognitive Computation, 2020.

[19] W. Sun et al. Multi-exit self-distillation with appropriate teachers. FITEE, 2024.

[20] S. Teerapittayanon et al. BranchyNet: fast inference via early exiting from deep neural networks. ICPR, 2016.

[21] B. Bross et al. Versatile Video Coding. IEEE TCSVT, 2021.

[22] F. Yang et al. Slimmable compressive autoencoders for practical neural image compression. CVPR, 2021.

[23] M. A. Y■lmaz et al. Slimmable video codec. CVPRW, 2022.

[24] A. Kuznetsova et al. The Open Images Dataset V4. IJCV, 2020.

[25] A. Mercat et al. UVG dataset: 50/120fps 4K sequences for video codec analysis. ACM MMSys, 2020.

[26] H. Wang et al. MCL-JCV: a JND-based H.264/AVC video quality assessment dataset. ICIP, 2016.